Neuro-Symbolic NLP: Taxonomy, Assessment, and Directions

Stergios Chatzikyriakidis [1,*], **Shalom Lappin** [2]

[1] *Centre for Linguistic Theory and Studies in Probability, University of Gothenburg;Department of Philology, University of Crete, Rethymno, Greece*
[2] *School of Electronic Engineering and Computer Science, Queen Mary, University of London, London, UK; Centre for Linguistic Theory and Studies in Probability, University of Gothenburg; Department of Informatics, King's College London*

Correspondence*:
Stergios Chatzikyriakidis
stergios.chatzikyriakidis@gu.se

ABSTRACT

Neuro-symbolic (NeSy) approaches promise to overcome the limitations of purely neural and purely symbolic NLP. In this paper we survey recent development in NeSy NLP and propose a system classification framework that combines Kautz's integration types with Lappin's injective-federative distinction. We then apply this taxonomy and show that even though federative architectures consistently outperform injective approaches, they remain underexplored. We assess performance across compositional generalization, reasoning, and robustness benchmarks, examine existing attempts at linguistic theory integration, and show how the taxonomy can guide future architectures that preserve the strengths of formal linguistic frameworks. We conclude with directions for scaling federative systems and exploring tighter integration.

Keywords: neuro-symbolic, NLP, compositionality, explainability, linguistic theory, hybrid models, reasoning, transparency

1 INTRODUCTION

It is undeniable that Neural models of language have achieved remarkable progress in the last decade or so. However, they still often rely on superficial patterns and also struggle with compositional generalisation and systematic reasoning. On the other hand, symbolic approaches while offering transparency and strong generalisation, lack scalability. *Neuro-symbolic* (NeSy) NLP promised to combine these strengths.

Different taxonomies have been proposed in an effort to organize the NeSy research landscape. Some focus on functional capabilities, classifying systems according to the research problems they address: knowledge representation, learning and inference, or explainability (Colelough and Regli, 2025). Others focus on architectural organization. Kautz (Kautz, 2022) proposes a six-type classification based on the depth of integration between neural and symbolic components. Lappin (2025) offers a simpler but architecturally grounded distinction between injective systems, which embed symbolic content within neural architectures, and federative systems, which maintain autonomous modules with structured interfaces. In this paper, we propose a unified classification framework that combines the existing taxonomies and survey the current landscape of NeSy NLP systems using this new combined taxonomy. we then examine

existing attempts at linguistic theory integration, show how the taxonomy can guide linguistically-informed architectures, and propose directions for developing foundational neuro-symbolic NLP.

1.1 Kautz's Taxonomy

Kautz (Kautz, 2022) proposes a six-way classification of NeSy systems based on the level of integration between the neural and the symbolic components. Type 1 systems are standard neural networks, in which the symbolic input is turned into vector representations for neural processing and then mapped back to symbolic output. Type 2 systems embed neural components within symbolic problem solvers. In these systems symbolic techniques invoke neural methods. AlphaGo is an example, where Monte Carlo tree search (symbolic) calls neural networks to evaluate game positions. In Type 3 systems, neural networks interpret perceptual data as symbols that are then processed by symbolic reasoners. Examples include the Neuro-Symbolic Concept Learner (Mao et al., 2019) and NS-VQA (Yi et al., 2018). Type 4 systems involve the use of symbolic reasoning to generate or label training data for neural networks, as in physics-informed neural networks. The last two types, Types 5 and 6, involve the most tightly connected systems. Type 5 systems like Logic Tensor Networks (Serafini and Garcez, 2016) and Neural Theorem Provers (Rocktäschel and Riedel, 2017) attempt bidirectional interaction.There, symbolic operations are represented as tensor operations that can be optimized via gradient descent. Type 6 represents true unification of neural and symbolic processing and remains largely aspirational. However, on a conceptual level, despite the move toward tighter integration, even the most advanced existing systems (Type 5) still maintain a conceptual distinction between the neural and the symbolic components. They treat them as distinct systems that must be connected in some way.

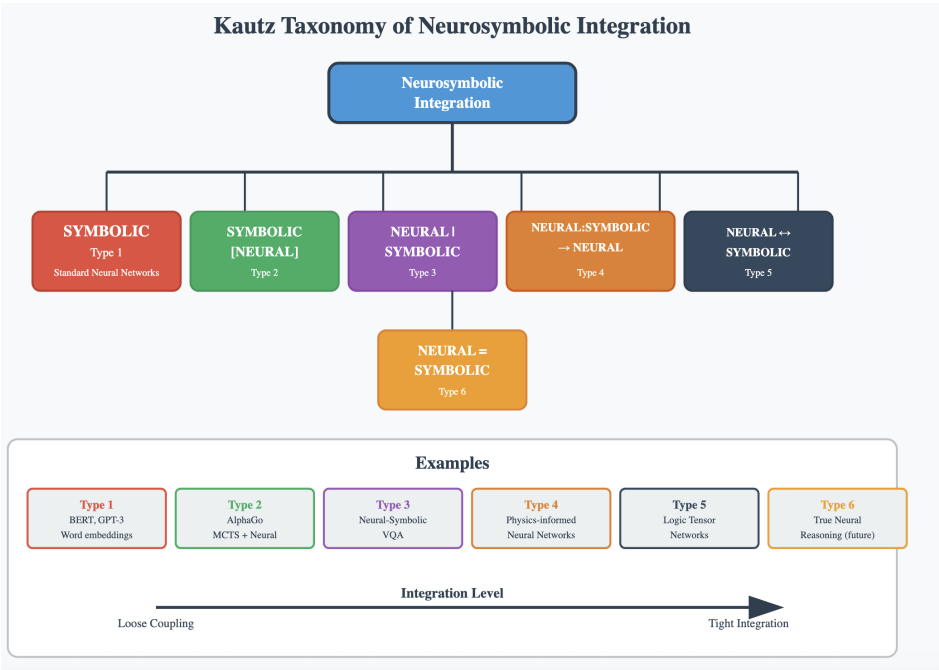

Figure 1. Kautz's taxonomy of neuro-symbolic architectures, showing the progression from loosely coupled systems (Types 1-2) to tightly integrated architectures (Types 5-6).

1.2 Lappin's Injective vs. Federative Distinction

Lappin (2025) makes a proposal that is based on a simpler distinction, is architecturally grounded, and cuts across Kautz's types. On the one hand, we have injective architectures that insert symbolic representations directly into the processing operations of a Deep Neural Network (DNN). This injection can happen in a number of ways: 1) via modifications in the architecture that incorporate symbolic bias in the computation, 2) using knowledge enrichment in which a symbolic system generates a biased training distribution (Hinton et al., 2015), and 3) via symbolic markers injected in the training. DNNs that incorporate syntactic structure into LSTMs (Tai et al., 2015; Bowman et al., 2016; Yogatama et al., 2017) or transformers (Bai et al., 2021; Sachan et al., 2021), and CNNs enriched with handcrafted knowledge features for domain-specific tasks (Zeng et al., 2014; Nguyen and Grishman, 2015) are some examples. On the other hand, a *federative* model combines a neural architecture with a symbolic module as part of the system, in which each module retains its independence. The neural component extracts features for an interface that labels them and then sends them to the logic component, thus, keeping the computational integrity of both systems (Cunnington et al., 2023).

Lappin shows an asymmetry w.r.t the performance of these two approaches in the taxonomy. On the one hand, injective models produce small performance gains over their non-enriched counterparts. To give an example, tree-enriched versions of BERT and RoBERTa show 1-2% accuracy increases on the GLUE benchmark (Bai et al., 2021), and enrichment with dependency graphs shows gains of 2.6-3.5 F1 scores for semantic role labeling, though these improvements require gold parses and largely disappear with predicted parses (Sachan et al., 2021). These already marginal gains seem to diminish further when DNNs receive more training data. Furthermore, the transparency claims often made for injective models also remain questionable, given that these systems still use non-linear activation functions (e.g. ReLU and softmax). On the other hand, Lappin notes, federative models show more substantial advantages in performance. For example, Cunnington et al.'s (2023) Feed Forward Neural-Symbolic Learner (FFNSL) presents considerable performance boost over non-symbolic baselines on difficult image classification tasks, requires substantially less training data to reach high accuracy, exhibits greater robustness under distributional shift, and generates transparent, inspectable rule-based hypotheses. Lappin further hypothesizes that this asymmetry between the two approaches (injective vs federative) may actually come from fundamental differences in how DNNs and symbolic systems represent patterns of regularity. It could be the case that DNNs may be unable to easily integrate symbolic components into their distributed representations (Smolensky, 1990).

1.3 Comparative Analysis

The two taxonomies, Kautz's on the one hand, and Lappin's on the other, focus on complementary aspects of NeSy systems. Kautz's taxonomy is mostly concerned with the mechanism of interaction between the two components, for example the way information flows between components, while Lappin puts the architectural location of symbolic content, i.e. whether it lives inside or outside the neural network at the core of the taxonomy. As already mentioned, there are other ways of organizing NeSy systems. Colelough and Regli (2025) propose a taxonomy that classifies NeSy systems based on the research problems they are called to tackle: knowledge representation, learning and inference, logic and reasoning, explainability and trustworthiness, and meta-cognition. Such classifications prove valuable to identify research gaps in what NeSy systems can do, but they provide limited guidance on how to design systems to achieve those capabilities. We believe that architectural taxonomies like those of Kautz and Lappin offer more direct design insights by revealing which integration methods produce gains, and which face architectural

limitations. The synthesis we propose aims to put together these architectural perspectives and, at the same time, remain grounded in specific challenges.

Table 1 presents the Chatzikyriakidis-Lappin (CL) taxonomy. It is a two-dimensional classification system combining Lappin's taxonomy with five interaction modes we have identified in the literature. Systems are organized by architectural location first (Injective vs. Federative), interaction mode second. In this way Kautz's types may appear non-consecutively. For example, Type 4 systems, i.e. systems in which symbolic reasoning generates or labels training data for neural learning, are split into I-T (symbols embedded then discarded), I-G (symbolic data generation for self-training), F-T (external symbolic preprocessing), and F-G (external symbolic data generation). Similarly, Type 3 systems, in which neural networks produce symbolic representations for symbolic reasoning, are split to I-R (internal conversion) and F-R (external reasoner). This non-sequential ordering is a choice, given that we privilege architectural location over the interaction mechanism. This is because we believe that Lappin's taxonomy reveals more about system performance than Kautz's integration depth. The first dimension of indicates architectural location (Injective or Federative), while the second interaction mode: **T**eaching (symbolic knowledge transferred then discarded), **C**onstraining (symbolic rules shape neural computation), **G**enerating (symbolic systems produce training data), **R**easoning (neural outputs feed symbolic inference), and **L**oop (dynamic bidirectional interaction).

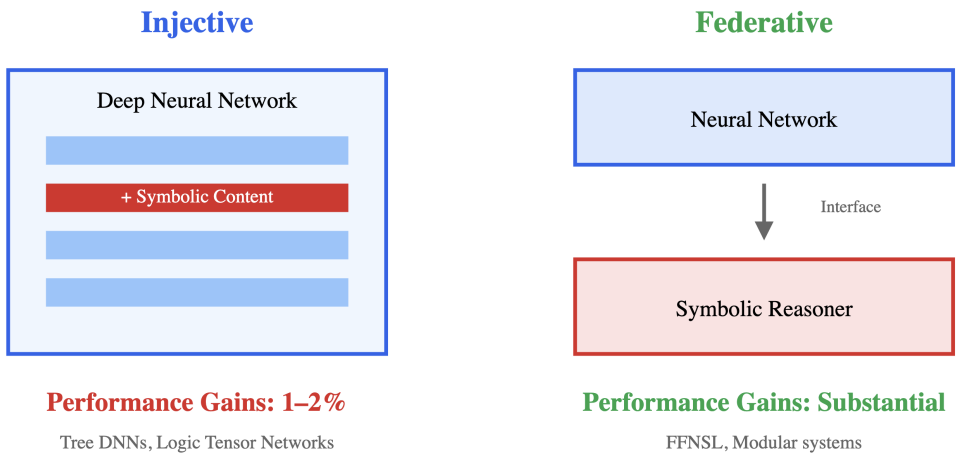

Figure 2. An overview of Lappin's taxonomy: On the left we see Injective systems (left) showing marginal gains (1-2%) and tight integration, while on the right we see federative systems showing substantial improvements via modular autonomy. The asymmetry might point to constraints on how the neural component absorbs symbolic knowledge.

The majority of existing NeSy work falls into the injective categories (I-*), i.e. to those employing various mechanisms of embedding the symbolic content within neural architectures. However, these are the approaches that, in the examples Lappin considers, show only marginal improvements, typically within the 1-2% accuracy range and further diminish with additional training data. Federative approaches, which, in the cases that he cites, indicate substantially stronger performance, are less explored, especially at foundational model scale. This state of affairs might point to the fact that the field has invested in architectural integration strategies that may face fundamental limitations, while leaving more promising paradigms underdeveloped. In Section 2, we use this taxonomy to systematically review major NeSy NLP systems of the last decade, documenting how architectural choices shape empirical performance.

Table 1. The CL system. It combines Lappin's architectural dimensions with Kautz's interaction patterns. Note that the Kautz column indicates approximate correspondences; see text for discussion of limitations.

CL Code	Related Kautz Type(s)	Description	Example System
Injective Hybrids (I-*): Symbols embedded within neural architecture			
`I-T`	Type 4	Symbolic knowledge distilled into neural weights, then discarded	Pre-training LMs with syntactic objectives
`I-C`	Type 5	Symbolic rules compiled into network architecture	Logic Tensor Networks
`I-G`	Type 4	Symbolic system generates data for neural self-training	Grammar-based data augmentation
`I-R`	Type 3	Internal neural representations converted to symbols for reasoning	Neural semantic parser + theorem prover
`I-L`	Type 5	Bidirectional symbolic-neural interaction within the model	Logical Neural Networks
Federative Hybrids (F-*): External symbolic modules interfacing with neural components			
`F-T`	Type 4	External symbolic preprocessing creates neural training data	Rule-based feature extraction + neural classifier
`F-C`	Type 2	External symbolic constraints guide neural processing	Constrained neural text generation
`F-G`	Type 4	External symbolic system generates training data	Knowledge graph-based dataset creation
`F-R`	Type 3	Neural frontend feeds external symbolic reasoner	NLProlog, DSR-LM
`F-L`	Type 2/3	Dynamic interaction between separate neural and symbolic modules	PAL, SatLM

Legend: I = Injective (symbols within neural architecture); **F** = Federative (external symbolic modules); **T** = Teaching; **C** = Constraining; **G** = Generating; **R** = Reasoning; **L** = Loop.

1.4 How to read the Chatzikyriakidis-Lappin (CL) code

A tag such as `F-L` has two characters:

- The **first** (`I` or `F`) indicates *where the symbols live*, i.e. inside or outside the model (Injective/Federative)
- The **second** (`T`, `C`, `G`, `R`, or `L`) describes the *interaction mode*, a categorization we propose based on patterns observed across the NeSy literature, loosely related to but not identical with Kautz's types.

For example, `I-T` refers to an injective model, in which symbolic training data are taught once then discarded, whereas `F-L` refers to a federative model involving a live bidirectional loop at inference time.

Of course, our taxonomy is not without limitations. We identify mainly two. First, the correspondence of interaction modes and the types proposed in Kautz is not exact. This is because Kautz's taxonomy measures integration depth, while we aim to capture information flow patterns. Furtermore, the mapping of the

two dimensions in the new taxonomy can also be difficult to make clear in some cases, particularly when models exhibit hybrid characteristics. Furthermore, we cannot adequately distinguish between the depth of integration. For example, an injective system that only adds syntactic features to input embeddings is not the same as a system that rebuilds its attention mechanisms based on type-theoretic constraints. However, the two systems receive the same classification in this taxonomy. Last, and perhaps most important, the taxonomy assumes a clear boundary between the symbolic and the neural component. However, systems implementing differentiable proof systems built directly into transformer blocks, or systems keeping hybrid representation spaces encoding both distributional and logical semantics fall into this category.

2 LANDSCAPE OF NEURO-SYMBOLIC NLP SYSTEMS

In this section, we review a number of major NeSy NLP systems. We organize the discussion according to the CL classification framework introduced in Section 1.3 and examine the way architectural choices (injective vs. federative) and interaction mechanisms shape system capabilities.

2.1 Injective Systems: Symbol Embedding in Neural Architectures

2.1.1 Injective-Constraining (I-C): Compiled Symbolic Structure

Injective systems embed symbolic constraints directly into the neural system. This is done via structural modification or differentiable logical operators.

Logic Tensor Networks (LTN) (Badreddine et al., 2022) encode end-to-end differentiable first-order logic via the integration of fuzzy logic directly into the computational graph, with predicates being represented as neural networks, constants as learnable embeddings, and logical connectives as differentiable fuzzy operators. LTN uses Real Logic, a differentiable form of first-order logic, to perform tasks such as multi-label classification and relational learning.

Riegel et al. (2020) propose **Logical Neural Networks** (LNN), a system, where neurons are directly mapped to weighted real-valued logic, allowing, at least theoretically, for bidirectional inference. However, when applied to question answering (Kapanipathi et al., 2021), the theoretical promise does not hold up completely, with the system seeing only small or moderate gains on a number of baselines (5.3% on Question Answering over Linked Data (QALD-9) and 16% on Large-Scale Complex Question Answering Dataset (LC-QuAD)). This is in line with Lappin's critique of injective approaches that argues that even if you compile symbolic constraints into the network, you are essentially guiding the learning process, and not fundamentally changing the distributed representations.

DeepProbLog (Manhaeve et al., 2021) integrates neural predicates into ProbLog's probabilistic logic programming framework. Neural networks act as predicates for perceptual tasks. DeepProbLog compiles the logic program into arithmetic circuits enabling the system to compute gradients efficiently. This achieves strong sample efficiency: the system needs just hundreds of training examples rather than the tens of thousands needed by pure neural baselines.

The performance increases show what is I-C's real potential: when logical structure is infused into the network, compositional generalization can emerge with only minimal examples, given that the composition rules are learned and not memorized. Additionally, for tasks like visual question answering, these systems are able to systematically generalize to novel combinations of learned concepts. However, the issue of scaling is still challenging. Given that the probabilistic inference overhead grows as program complexity grows, these systems are limited to small-scale problems.

Scallop (Li et al., 2023) presents an I-C integration infrastructure using a Datalog-based NeSy programming language that further enables differentiable reasoning. Tree-LSTMs (Tai et al., 2015) use syntactic structure via the incorporation of parse trees into the LSTM. The system performs well on benchmarks where syntax is a key predictor, such as the Stanford Sentiment Treebank 5-class (SST-5) benchmark (51.0% vs. 46.4%). However, it falls behind on tasks where syntactic structure offers less leverage. Syntax-aware transformers (Bai et al., 2021; Sachan et al., 2021) show a similar picture with a 0.3-2.2% increase on various benchmarks, thus following the general pattern. Neural Logic Machines (Dong et al., 2019) learn first-order logic rules via neural tensor operations. They manage to get perfect generalization on synthetic reasoning tasks (family trees, sorting, shortest paths). Despite this, the system remains limited to domains with explicit logical structure.

Generalizing, one could argue that for I-C systems, symbolic structures provide decisive advantages w.r.t. compositional generalization, in which explicit structure directly encodes the target abstraction. We have seen cases where this is true e.g. DeepProbLog's order-of-magnitude sample efficiency and CPG's 1000× on SCAN. At the same time, this type of systems provide only small improvements (1-2%) on more general NLU tasks, where symbolic constraints compete with the network's statistical shortcuts.

2.1.2 Injective-Teaching (I-T): Distilled, then discarded symbolic knowledge

Systems of this type insert symbolic knowledge in neural weights during pre-training, and then lose the symbolic component. This means that the advantages symbolic structure has to offer early in training disappear as the optimization continues.

Syntax-aware pre-training objectives, such as those applied to variants of BERT and RoBERTa, are examples of such architectures. The accuracy gains are modest with Tree-enriched BERT variants showing between 1 and 2% accuracy increase on GLUE (Bai et al., 2021). Furthermore, given the nature of the architecture, these already small increases often disappear when additional training data are given to the models. This suggests that a symbolic backbone can help early in the training process as it provides useful inductive biases, but, nonetheless, it fails to provide a persistent architectural advantage. The reason is that gradient descent will eventually discover equivalent representations via purely distributional patterns, making the initial symbolic injection irrelevant.

I-T systems are thus distinct from other injective approaches architecturally, given that they abandon the symbolic component entirely after training has been done. This is in contrast for example to I-C systems that maintain compiled symbolic constraints throughout deployment, or I-R systems that generate symbolic representations at inference time.

2.1.3 Injective-Reasoning (I-R): Internal NeSy Conversion

In this category, we find architectures in which neural networks produce intermediate symbolic representations, that are processed via reasoning mechanisms in a unified, end-to-end fashion. The **PRover** (Saha et al., 2020) is a good example of such architecture. The transformer in this system predicts both the answers and the proof graphs. Neural components generate the nodes and edges that represent facts, rules, and logical dependencies. These attributes allow **Scallop** (Li et al., 2023) to utilize a Datalog-based language for differentiable reasoning within I-C integration. **Tree-LSTMs** (Tai et al., 2015) take a different approach by embedding syntactic parse trees into the model state. This design improves performance on syntax-heavy tasks like SST-5 (51.0% vs. 46.4%), but the benefit is lost when grammar is not a primary predictor. **Syntax-aware transformers** (Bai et al., 2021; Sachan et al., 2021) mirror this pattern, yielding only marginal gains between 0.3% and 2.2%. **Neural Logic Machines** (Dong et al., 2019) use

tensor operations to learn first-order logic, reaching perfect scores on synthetic tasks like sorting or path finding while remaining restricted to rigid logical domains. Although differentiability facilitates the joint optimization of perception and reasoning, these systems still lack true symbolic interpretability. Their representations are learned latently and prioritize correlation with training labels over any strict adherence to logical truth.

Other I-R systems are similar in this manner. For example, **ProofWriter** (Tafjord et al., 2021) uses T5 to iteratively produce single-step implications, gathering symbolic proof chains in one unified architecture. **RNNLogic** (Qu et al., 2021) makes use of logic rules as latent variables. An RNN-based generation system produces candidate rules and a reasoning module evaluates them. Rules are generated internally, the model uses them to reach an answer, but these rules are not shown to the user.

These systems allow for the joint optimization of perception and reasoning, which preserves differentiability, but they lack real symbolic interpretability. The representations are only learned latently. This means the model optimizes for correlation with training labels rather than following any actual logical truth. This puts I-R systems in an awkward position, since they suffer from the high computational cost of maintaining symbolic structures, but lack the transparency of external reasoners. It is unsurprising, then, that F-R systems that involve a cleaner architectural separation, tend to supersede them.

2.1.4 Injective-Loop (I-L): Bidirectional Internal Interaction

The I-L is arguably the most ambitious injective approach, as it strives for bidirectional information flow between the neural and the symbolic components in a unified, trainable architecture. Think of it as trying to get the best of both worlds by having symbols and neurons talk to each other continuously during both training and inference.

NeurASP (Yang et al., 2020) takes a different route and embeds neural networks into Answer Set Programming. Neural perception is informing symbolic rules, and a number of symbolic constraints guide neural learning, all in a unified optimization loop. **Differentiable Neural Computer** (Graves et al., 2016) maintains both distributed and symbolic-like memory representations with bidirectional read-write operations.

The computational challenges involved in maintaining a tight bidirectional coupling between the two components have proven difficult to overcome as the scale of the data and the task increase. As such, it comes as no surprise that large-scale NLP implementations in this category remain rare. These systems work nicely on small, structured problems, but moving to foundation-model size is very difficult to achieve, as overhead grows faster than the actual benefits, and the optimization becomes unstable.

2.2 Federative Systems

2.2.1 Federative-Reasoning (F-R)

Probably the most successful category of federative systems pair the neural component with an external symbolic reasoner. A clear separation of the two components exist and neither component compromises its computational identity.

An example of such a system is **NLProlog** (Weber et al., 2019). In this system, sentences from a knowledge base are embedded via Sent2Vec. Then, when the system is asked to provide an answer, a Prolog theorem prover with weak unification searches for logical derivations. However, instead of the exactness of symbolic pattern matching, NLProlog uses nearest-neighbor lookup to discover relevant facts.

In this way, the system is able to deal with the noise and variation of language, and yet remain logically sound.

On MedHop, a dataset that tests multi-hop reasoning abilities, NLProlog achieves 29.3% accuracy on hidden test, as well as fully interpretable proof traces. This means that one can see exactly the facts the system retrieved and the way these were chained together to reach a conclusion, something which is not possible in pure neural approaches.

DSR-LM (Zhang et al., 2023) builds on this architecture and provides even better results. The system uses a language model for perception with weighted rule-based reasoning. The neural component deals with natural language, extracting symbolic predicates, and then the symbolic component applies a number of learned weighted rules in order to deal with deduction. The system surpasses pure neural baselines by more than 20% on tasks that require deductive reasoning. The clear separation of the two modules lets each one do what it does best, i.e., the LM deals with the noise and variation of linguistic input and the symbolic module handles inference.

Other F-R Systems. Logic-LM Pan et al. (2023) maps natural language to FOL or SAT constraints for solver refinement, boosting performance by 39.2% over standard prompting. LINC Olausson et al. (2023) shows a comparable pattern. In that work, a StarCoder-15B model integrated with a first-order logic prover beats GPT-4 on ProofWriter by 10%. Greedy NTP Minervini et al. (2020) addresses link prediction using a hybrid of k-NN search and Prolog-style backward chaining. This setup achieves competitive WN18 accuracy, but it operates at speeds exceeding symbolic baselines. Current research also focuses on applying these structures to complex geometry and proving. It operates at significantly higher speeds than typical symbolic baselines. Beyond these applications, recent work has begun to target the specific hurdles of complex geometry and theorem proving. It applies these neuro-symbolic techniques to the challenges of complex geometry and automated theorem proving. For example, Alpha Geometry (Trinh et al., 2024) combines a neural model for suggesting constructions with a symbolic deduction engine. In LEGO-Prover, Wang et al. (2024a) follow a different route. They use libraries of verified lemmas, resulting in the improvement of neural theorem proving on the miniF2F validation set by 9.0% (4.5% on test)."

The pattern across F-R systems is clear and consistent. For tasks requiring a separation of perception from logical inference, federative systems present clear advantages (20%+ for DSR-LM, interpretable proofs for NLProlog). The architectural boundary emerges as a strength, given that each component can be optimized independently, verified separately, and also understood in isolation.

2.2.2 Federative-Loop (F-L): Dynamic Iterative Refinement

F-L architectures include federative systems that involve dynamic interaction between separate neural and symbolic modules during inference. These systems are also bidirectional in that the neural output informs symbolic reasoning, and symbolic results guide the neural generation.

Symbolic Working Memory (Wang et al., 2024b) is an example of such an approach that is used for multi-step deductive reasoning. The system includes an external memory module that stores facts and rules in dual format (natural language and symbolic). When reasoning about a problem, the neural component parses the question and queries the memory for relevant information. Retrieved facts are converted to symbolic form, the symbolic reasoner performs rule grounding, and the results feed back to the neural component, which may query the memory again or generate intermediate conclusions.

This process continues until an answer is produced. In each step the symbolic representations in memory get updated, allowing the construction of chains of reasoning in an incremental fashion, instead of trying

to deal with everything in one pass. The fact that the system involves a clear separation between neural understanding (which facts are relevant) and symbolic reasoning (how to connect these facts) has the welcome benefit that the components can be inspected and corrected independently.

F-L represents the most sophisticated federative architecture, as it involves adynamic interaction between separate neural and symbolic modules at inference. This is not just one-way information flow, but a loop in which the neural output informs the symbolic component. Similarly, symbolic results guide neural generation, with the process iterating towards convergence.

Other F-L Systems. Iterative refinement is also central to work like **Natural Language Embedded Programs (NLEP)** (Zhang et al., 2024), which targets formal reasoning. It generates and then executes Python programs step-by-step, improving interpretability. This code-driven approach echoes **Program-Aided Language models (PAL)** (Gao et al., 2023), where offloading reasoning into code improved over **PaLM-540B** on GSM8K by 15%. **Satisfiability-Aided Language Models (SatLM)** (Ye et al., 2023) takes this further and applies it to constraint satisfaction problems, using declarative constraints for SAT/SMT solvers, improving upon program-aided LMs by 23%. Most recently, **Symbolic Verification** systems (e.g. **Logic-Scaffold**) (Quan et al., 2024) have closed the loop completely. They use external verifiers that check coherence in LLM-generated specs, with the resulting feedback fed back to shape subsequent corrections. F-L architectures have also proven effective for phonologically-grounded rhyme generation. Chatzikyriakidis and Natsina (2025) present a NeSy system for poetry written in Modern Greek that combines LLMs and a deterministic phonological engine in a Generate-Verify-Refine loop. Pure LLM generation achieves under 4% validity for phonologically correct poems, whereas the verification loop raises this to 73.1%. This makes clear that iterative symbolic feedback is effective, in cases where phonological constraints are needed. F-L systems shine when tasks call for multi-step verified reasoning. This is to be expected given their self-correcting ability. The capacity to reason across iterations lets them tighten their logic and clean up symbolic representations, which single-pass I-R and F-R models are unable to do. Naturally, there is a cost. Running multiple neural-symbolic round trips proves computationally expensive. But, when moving to critical domains, F-L architectures offer the necessary balance of capability and verification.

2.2.3 Federative-Generating (F-G): External Symbolic system generating training data

In F-G architectures, one finds symbolic modules as training data generators for neural components, i.e. they involve a symbolic generator that makes sure the generated data satisfies formal requirements. The neural component then learns from this verified dataset.

NSDG (Neuro-Symbolic Data Generation) (Li et al., 2024) is a system in this category that converts math problems to SMT-LIB (Satisfiability Modulo Theories Library) format (the standard format used for writing logic formulas for SMT solvers), and it mutates them systematically. These mutations preserve logical correctness, and at the same time are able to create diverse training instances. Examples in the resulting datasets have formal guarantees of logical validity. This is very useful for domains in which training data might be scarce, but formal specifications are available.

F-G and I-T systems are foundationally different in the way symbolic knowledge post-distillation is handled. I-T models discard the symbolic component after pre-training, but F-G models keep the generator active during the whole process. This allows for the on-demand synthesis of new training examples, a vital feature for adapting to shifting problem distributions. Current dataset expansion methods rely on neural techniques like back-translation or generative paraphrasing, which often produce plausible but strictly unverified results. F-G offers an alternative by ensuring that all augmented data meets formal,

domain-specific requirements. Choosing these systems over neural methods is a matter of prioritizing correctness over coverage.

2.2.4 Federative-Constraining (F-C): External Guidance

F-C architectures function as post-hoc filters, using symbolic constraints to govern inference without impacting the training process. This pipeline uses a neural model in order to generate potential outputs which a symbolic module then validates or ranks using formal logic. A standard implementation of this is **NeuroLogic Decoding** (Lu et al., 2021); it prunes a language model's output space using predicate logic specifications that are entirely absent during the training stage. Such total architectural separation restricts interaction to output-level filtering decisions. The primary advantage here is implementation simplicity: layering a symbolic checker over a frozen, pretrained model yields formal output guarantees without requiring any weight updates.

However, F-C systems lack the iterative refinement that one finds in F-L architectures. The neural model generates and the symbolic module accepts/rejects. There is no feedback loop, and no opportunity for the symbolic constraints to guide the neural generation process toward better candidates. This limits the system's effectiveness when the constraint satisfaction score is low, because the system has to generate many candidates to find acceptable ones. As already mentioned, this category remains minimally explored compared to other approaches like F-R and F-L. The reason for this might be that the benefits of one-shot are weaker than iterative refinement or integrated reasoning.

2.3 Recent LLM-Era Developments: Hybrid Patterns

The distinction between injective and federative systems is becoming increasingly blurry, with recent integrations involving architectures that look federative by design but behave like tight feedback loops in practice. **Logic-LM, LINC, and PAL** illustrate this Neural→Symbolic grey area. In these systems, the LLM drafts a formal spec (SAT, SMT, or Python) and an external solver executes it. Strictly speaking, this is a federative system, because the solver remains a distinct, external entity. However, the coupling during inference is very tight. The cycle of generating a spec, executing it, and using the results to drive the next step mimics an F-L loop far more than it does standard, one-shot F-R reasoning.

DeepSeek-R1 (Guo et al., 2025) challenges the taxonomy in a different way. Extended reasoning emerges from pure reinforcement learning with no fine-tuning. DeepSeek-R1 used only GRPO (Group Relative Policy Optimization) during its training phase that involved no human-annotated reasoning examples. However, it managed to develop self-verification, reflection, and long chain-of-thought reasoning, achieving 79.8% on AIME 2024 (American Invitational Mathematics Examination), slightly improving upon OpenAI o1-1217.

The ability to transfer knowledge in these models is remarkable. Open-source distilled models show that reasoning patterns can effectively transfer to smaller systems. These systems resist categorization. They're primarily neural with no explicit symbolic component. However, they seem to involve emergent symbolic-like reasoning via test-time compute scaling. One way to conceptualise them is as implicit I-L systems in which symbolic structure emerges from optimization rather than being explicitly compiled. Alternatively, the taxonomy doesn't capture what's happening here.

Large Concept Models (Barrault et al., 2024) push beyond current taxonomic boundaries in a different direction. They operate on sentence-level SONAR (Sentence-Level multimOdal and Language-Agnostic Representations) embeddings, not tokens. The 7B parameter model trained on roughly 2.7 trillion tokens

demonstrates strong zero-shot cross-lingual generalization. Processing at the conceptual level seems to provide advantages for reasoning and long-document understanding.

However, LCM lacks formal symbolic integration, given that it operates in entirely learned embedding spaces. Thus, there is no explicit logic, no theorem provers, or formal rules. It adapts an architecture where multiple abstraction levels (tokens, sentences, concepts) coexist. One could imagine symbolic constraints in these systems operating at the higher conceptual levels, while neural processing handles the lower-level perceptual mapping. This "hierarchical neuro-symbolic integration" represents a design space for which current taxonomies do not yet have a dedicated category.

2.4 Compositional Generalization

The SCAN benchmark is a good resource to test the compositional ability of various models , as it tests how the models are able to map instructions in natural language form to actions not seen during training. On these tasks,

Transformers fail. Lake and Baroni (2018) report that seq2seq models achieve only around 20%

accuracy on length-based splits, with some configurations reaching near 0%. Modification of the

training data can boost scores to near-perfect levels. However, this method sidesteps rather than solves the actual architectural problem (Patel et al., 2022). NeSy systems provide a quite different result. The Compositional Program Generator achieves 100% accuracy on all SCAN test splits using just 14 training examples (Klinger et al., 2023). In comparison, the neural baseline requires nearly 17,000 examples.

The CPG learns composition rules rather than memorizing combinations. Given the rule for how "jump" combines with "twice", it generalizes to "run twice", "walk twice", anything else combined with "twice". Neural models have to see each combination. Lake and Baroni (2023) show similar results with MLC (Meta-Learning for Compositionality). The system reaches human-level systematic generalization on instruction-learning benchmarks, and manages to outperform frontier LLM models like GPT-4.

COGS (Compositional Generalization Challenge) exhibits the same pattern, with seq2seq models achieving 96-99% in-distribution performance, but they reach only 16-35% generalization accuracy (Kim and Linzen, 2020). They are able to memorise the training distribution effectively, but they are not able to generalise. The AM Parser, a NeSy compositional semantic parser with I-C architecture, reaches over 98% generalization accuracy. CPG hits 100% using only 22 examples (Klinger et al., 2023).

Neural approaches specifically adapted for the task can reach up to 88% on specific splits, but they require full training sets and lack systematic compositionality. They're fitting the test distribution, not learning compositional principles.

CFQ (Compositional Freebase Questions) presents a more nuanced picture. Standard Transformer models achieve 5-29% accuracy (mean 18%) on Maximum Compound Divergence splits (Keysers et al., 2020) and pre-trained models like T5-11B reach 40.9%. Lastly, specialized architectures such as Hierarchical Poset Decoding

achieve 67% (Guo et al., 2020). There's a strong negative correlation between compound divergence and accuracy. In general, models tend to perform better when train and test distributions share compositional patterns. They fail on novel combinations.

As a general conclusion, we could say that for tasks needing some sort of systematic compositional generalization, I-C architectures with explicit compositional structure seem to excel. The 1000× sample

efficiency comes from learning composition rules rather than memorizing combinations. But this advantage seems limited to domains with explicit compositional structure involving commands, semantic parsing, query generation. Other aspects of natural language processing may not benefit equally. When the compositional structure isn't explicit or the task requires understanding beyond syntactic combination, the advantage disappears.

2.5 Reasoning and Verification

Logical reasoning benchmarks reveal mixed success for Neural and NeSy systems, with performance gaps that appear less extreme than those seen in compositional generalization. The PRover system (Saha et al., 2020) achieves 87% accuracy on rule-based reasoning, yet this figure sinks to 65% when five-step reasoning chains are required. Such a decline illustrates how compounding errors consistently derail deeper inference. LogiQA 2.0 (Liu et al., 2023) presents a similarly difficult hurdle, testing categorical and conditional logic across 35,000 premise-hypothesis pairs. Top-performing models typically perform between 48% and 59% accuracy, trailing human benchmarks by a wide margin.

FraCaS is a dataset of 346 logical Natural Language Inferenced (NLI) cases that has been developed to check NLI systems' ability to reason logically. In this dataset,symbolic systems outperform neural ones. For example, type-theoretical systems achieve 81%(Bernardy and Chatzikyriakidis, 2021), while GPT-4 reaches 75% (Teng et al., 2023) and BERT-based models fall below 55% on monotonicity reasoning (Yanaka et al., 2019). However, while the 346-problem benchmark might enable targeted semantic evaluation, it is fairly limited given its very small size.

Vulnerabilities become more obvious under adversarial pressure; for instance, RoBERTa's accuracy on the Adversarial NLI (ANLI) dataset collapses from 73.8% in Round 1 to 44.4% by Round 3 (Nie et al., 2020). To mitigate this, **VAULT** (Kazoom et al., 2025) utilizes automated, validated adversarial generation to achieve non-trivial gains, pushing RoBERTa-Base to 80.95% on ANLI and boosting MultiNLI performance by 17.32%. Contrast test sets similarly expose model brittleness; systems maintaining 85-95% accuracy on standard benchmarks often see 5-25% performance drops following minimal perturbations (Gardner et al., 2020). As a case in point: ELECTRA-small slides from 89.9% to 72.5% on SNLI contrast sets, though contrast-augmented training can restore it to 85.5% (Sanwal, 2024). Such data indicates that models prioritize superficial pattern memorization over robust reasoning. Currently, neither injective nor federative architectures provide a clear advantage against adversarial or contrast challenges, as both paradigms are susceptible to spurious correlations. Architectural choice appears secondary to the training objective in these scenarios, implying that while the injective-federative split clarifies compositional generalization, is less useful for addressing robustness in distribution shift.

2.6 Taxonomy-Revealed Patterns

Our classification system reveals a number of patterns across architectures. First, I-C systems dominate w.r.t. research volume, but they achieve small performance gains (1-2%) on most tasks. The exception seems to be compositional generalization benchmarks where explicit structure provides decisive advantages (1000× sample efficiency on SCAN/COGS). F-R systems, on the other hand, present good performance increases (20%+ on deductive reasoning for DSR-LM), especially for tasks in which a separation of perception from logical inference is helpful. F-L systems, i.e. systems that combine LLMs with external verifiers, constitute a promising direction. None of these systems have been produced at foundational scale.

I-T systems are ephemeral, given that the advantage of symbolic knowledge distilled during pre-training disappears with continued training (thus, no ongoing advantage). I-R systems are computationally expensive

and superseded by models with cleaner F-R separation. F-G systems are symbolic data generators that are promising but underused compared to neural augmentation strategies.

Lastly, our taxonomy reveals a research gap. Federative systems are more successful but less represented. The field seems to be investing in marginal-gain injective approaches (I-C systems showing 1-2% improvements), leaving stronger federative paradigms underexplored.

This imbalance exists partly because of skepticism, but mostly because the field simply hasn't tested federative architectures at the scale of foundation models. Section 4 examines whether this gap stems from fundamental constraints or simply insufficient research investment, and it considers integrated systems that transcend current taxonomic boundaries.

3 NEURAL-SYMBOLIC NLP AND LINGUISTIC THEORY

Linguistic theory remains a secondary concern for the systems detailed in Section 2, regardless of their technical merit. Optimization pressures frequently help neural networks bypass structural features, such as tree masks or dependency attributes, in favor of raw statistical shortcuts (Gururangan et al., 2018). This preference for dataset artifacts over principled compositional mechanisms highlights a persistent reliance on spurious correlations (Baroni, 2020). While probing studies investigate quantifier scope and monotonicity, they operate as post-hoc analyses that neither constrain model design nor contribute to theoretical linguistics. Recent analyses of deep learning's relationship with linguistic representation document this widening gap between neural advances and theoretical understanding (Lappin, 2021; Linzen and Baroni, 2021; Pavlick, 2022).Neither paradigm learns from the other.

This stalemate shows a number of gaps in contemporary NLP. All NeSy models we have today are bespoke, as they are designed for narrow tasks like visual reasoning or knowledge graph completion (Yi et al., 2018; Sarker et al., 2021). As such, they do not fulfill the foundational role that BERT or GPT play in purely neural NLP, i.e. a single pre-trained core adaptable system across diverse tasks with minimal fine-tuning. Moreover, linguistic theories are never pitted against each other under large-scale conditions. NLP imports isolated cues while calling results "linguistically informed"; theorists validate rules on carefully hand-crafted sentences. What is lacking is any mechanism for theories to reconfigure in response to large-scale evidence, or for neural systems to systematically incorporate theoretical insights (Linzen and Baroni, 2021; Pavlick, 2022). Finally, learning algorithms are conspicuously absent from mainstream formal linguistics. The disconnect between technical complexity and linguistic grounding in Section 2 remains a persistent issue. Even when architectures incorporate structural constraints, such as tree masks or dependency features, networks frequently abandon these properties during optimization in favor of raw statistical shortcuts (Gururangan et al., 2018). This tendency to exploit dataset artifacts and spurious correlations actively hinders the development of principled compositional mechanisms (Baroni, 2020). While probing studies have successfully identified model behaviors regarding quantifier scope and monotonicity, they operate primarily as post-hoc observations. Because they do not impose design-level constraints, these findings fail to either inform theoretical linguistics or reshape the underlying model architectures.

3.1 Existing Attempts at Linguistic Theory Integration

Work in which linguistic theories are combined with neural systems is sparse. However, a couple of proposals have been put forth. Perhaps, the most substantial one is Kogkalidis et al.'s work on Neural Proof Nets (Kogkalidis et al., 2020). There we find an implementation of differentiable proof nets for type-logical

grammar using Sinkhorn networks. This system achieves end-to-end parsing from raw text to λ-calculus terms, with the proof-theoretic structure of linear logic preserved throughout. The architecture is genuinely NeSy (F-R), in that the neural components handle supertagging and axiom link prediction, and proof nets make sure that outputs are well-formed derivations with compositional semantic interpretations.

Other work integrates linguistic formalisms less deeply. For example, in Clark (2021); Tian et al. (2020) we find neural Combinatory Categorial Grammar (CCG) parsing, which is a combination of transformer-based supertaggers with symbolic CCG combinators. It produces SoA parsing accuracy results and preserves the transparent syntax-semantics interface of CCG. Zhou and Zhao's present an HPSG neural parser (Zhou and Zhao, 2019), that introduces head-driven phrase structure into joint constituent-dependency parsing. This is an I-C system, in which the linguistic formalism informs architectural design rather than participating in inference. Monroe et al. (2017); Cohn-Gordon et al. (2018) propose systems that combine Bayesian pragmatic reasoning with neural models, using Rational Speech Acts (RSA) as differentiable layers, an I-L architecture with bidirectional flow between neural and probabilistic-symbolic components. Neural semantic parsing to λ-calculus (Dong and Lapata, 2016; Jia and Liang, 2016) produces Montague-style logical forms, but it treats them as output targets rather than integrating compositional semantics into the reasoning process.

3.2 The CL Taxonomy as a Guide for Linguistically-Informed Architectures

Neural integration of major linguistic frameworks largely does not exist. The work in Section 3.1 covers most of what has been attempted. We believe the taxonomy proposed in this paper can function as a guide for future work in this direction, helping to match theoretical frameworks to an appropriate NeSy architecture A few examples illustrate the approach.

Type-theoretic semantics is based on type checking and proof construction, i.e. operations that can easily be fed to external proof assistants like Coq, Agda, Lean. An F-R architecture is an obvious choice here: neural components parse and extract semantic roles and the type checker verifies well-formedness An F-L variant would involve a system where type-checking failures trigger neural reanalysis.

Incremental frameworks like Dynamic Syntax (Kempson et al., 2001) are a good fit for an F-L architecture, in which the neural part can process the next word, while the symbolic part performs the necessary steps for tree growth, and tree constraints in turn can shape neural predictions for what comes next.

Non-compositional frameworks like Construction Grammar (Goldberg, 1995) that use form-meaning pairs to organize knowledge are a better fit for an F-G architecture. Symbolic constructions are used to generate training data with guaranteed grammatical properties.

Recapitulating, we suggest to identify the framework's main contribution (verification, incrementality, productivity), and then pick the CL category that preserves it. This is a practical suggestion that can, in principle, help in embedding linguistic theories as symbolic modules of NeSy systems, but support the choice of those parts of linguistic theories researchers regard as most significant for NeSy systems, or the specific tasks that NeSy systems are asked to solve.

4 TOWARD FOUNDATIONAL NEURAL-SYMBOLIC NLP

The taxonomic analysis in Section 2 shows that federative architectures consistently outperform injective ones, but the field keeps investing in injective designs to a greater extent than federative ones. These popular injective approaches deliver small gains, 1-2% improvements, that often vanish with more training data.

Furthermore, integration of linguistic theory across all methods is almost non-existent. Given this situation, we propose two research priorities.

First, federative architectures should be scaled up to foundation-model size. Right now, every successful federative system is small scale. FFNSL relies on modest neural components. DSR-LM and NLProlog tackle specialized reasoning tasks. Planning systems confined to structured action domains. We do not know whether keeping components separate still makes sense at a billion parameters. The advantages visible at small scale, modular transparency, verified reasoning, specialized components, may or may not carry over. Interface overhead could balloon. Symbolic reasoners cut off from neural representations might struggle to learn. These are empirical questions that must be experimentally explored.

Second, suppose scaled federative approaches do work. The architectural divide that defines federation could still cap what neuro-symbolic integration achieves. Taxonomies available now, Kautz's interaction types, Lappin's architectural categories, the CL framework proposed here, all draw sharp lines between neural and symbolic sides. But some systems go beyond that line, e.g. attention mechanisms with types built in, proof obligations threaded via gradient descent, unified substrates where representations mix distributional and logical properties from the start. Do these tightly coupled designs surpass federative ones in practice? We do not have the evidence to answer this question. Pure neural systems keep failing in familiar ways: hallucinating due to finite representational capacity (Peng et al., 2024), losing compositionality as complexity rises (Xu et al., 2024). Small fixes do not seem to help. Whether deeper integration addresses these failures or creates different problems must be tested.

Work across the model categories from Section 2 should clarify which architectural models will be fruitful. The field needs federative systems built at scale, empirical checks on whether their advantages survive, and frank assessments of where architectural boundaries help and where they interfere. The taxonomic analysis here indicates what succeeds at small scale and where research effort might not have produced the desired results. Building foundation-scale systems based on these findings is the required next move.

5 CONCLUSION

In this paper, we propose the Chatzikyriakidis-Lappin (CL) classification system (Table 1), that combines Lappin's architectural dimension along with five interaction modes of information flow (Teaching, Constraining, Generating, Reasoning, Loop) to systematically classify NeSy systems. The proposed framework shows an informative pattern already present in Lappin's taxonomy, namely that federative systems (F-R, F-L) consistently outperform injective approaches (I-C, I-T), but injective systems dominate published research. The CL taxonomy also reveals underexplored combinations. For example, the F-C category remains nearly empty, while I-L lacks large-scale implementations. The taxonomy has limitations. Systems with deeply integrated architectures where the neural-symbolic boundary dissolves entirely resist classification. It is true that deeply integrated systems exist as proposals. However, whether they offer any advantages over federative architectures remains an open question. Two research priorities emerge from our paper. First, the field should systematically scale federative architectures to foundation-model size in order to test whether advantages observed at small scale persist. Second, the architectural boundaries that define current taxonomies may themselves constrain what NeSy integration can achieve. The question of whether tighter integration helps or simply introduces new problems requires systematic investigation. Future work should determine which approaches prove viable at scale. The field needs implementations, empirical tests, and honest evaluation of where architectural choices matter and where they don't.

CONFLICT OF INTEREST STATEMENT

The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

AUTHOR CONTRIBUTIONS

SC and SL jointly conceived and written all sections of the paper. Both authors reviewed and approved the final manuscript.

FUNDING

ACKNOWLEDGMENTS

AI USE DECLARATION

Generative AI (Claude 4.5-opus) was used to check for grammar and syntax issues of the paper. The same AI model system helped in the creation of Figure 1 and Figure 2 (the model was prompted to generate an svg file with the specific info to be included and then the svg code was corrected by the first author to reach the desired output

DATA AVAILABILITY STATEMENT

All resources referenced in this article are publicly available. Scripts and supplementary materials can be found at [URL].

REFERENCES

Badreddine, S., Garcez, A. d., Serafini, L., and Spranger, M. (2022). Logic tensor networks. *Artificial Intelligence* 303, 103649

Bai, J., Wang, Y., Chen, Y., Yang, Y., Bai, J., Yu, J., et al. (2021). Syntax-BERT: Improving pre-trained transformers with syntax trees. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, eds. P. Merlo, J. Tiedemann, and R. Tsarfaty (Online: Association for Computational Linguistics), 3011–3020. doi:10.18653/v1/2021.eacl-main.262

Baroni, M. (2020). Linguistic generalization and compositionality in modern artificial neural networks. *Philosophical Transactions of the Royal Society B: Biological Sciences* 375, 20190307. doi:10.1098/rstb.2019.0307

Barrault, L., Duquenne, P.-A., Elbayad, M., Kozhevnikov, A., Alastruey, B., Andrews, P., et al. (2024). Large concept models: Language modeling in a sentence representation space. *CoRR*

Bernardy, J.-P. and Chatzikyriakidis, S. (2021). Applied temporal analysis: A complete run of the FraCaS test suite. In *Proceedings of IWCS*. 11–20

Bowman, S. R., Gauthier, J., Rastogi, A., Gupta, R., Manning, C. D., and Potts, C. (2016). A fast unified model for parsing and sentence understanding. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Berlin, Germany: Association for Computational Linguistics), 1466–1477. doi:10.18653/v1/P16-1139

Chatzikyriakidis, S. and Natsina, A. (2025). LLMs Got Rhythm? Hybrid Phonological Filtering for Greek Poetry Rhyme Detection and Generation. *arXiv preprint arXiv:*

Clark, S. (2021). Something old, something new: Grammar-based CCG parsing with transformer models. *arXiv preprint arXiv:2109.10044*

Cohn-Gordon, R., Goodman, N., and Potts, C. (2018). Pragmatically informative image captioning with character-level inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. 439–443

Colelough, B. C. and Regli, W. (2025). Neuro-symbolic AI in 2024: A systematic review. In *Proceedings of the Logical Foundations of Neuro-Symbolic AI Workshop (IJCAI 2024)* (Jeju Island, South Korea), vol. 3819 of *CEUR Workshop Proceedings*

Cunnington, D., Law, M., Lobo, J., and Russo, A. (2023). FFNSL: Feed-forward neural-symbolic learner. *Machine Learning* 112, 515–569. doi:10.1007/s10994-022-06278-6

Dong, H., Mao, J., Lin, T., Wang, C., Li, L., and Zhou, D. (2019). Neural logic machines. In *7th International Conference on Learning Representations, ICLR 2019* (New Orleans, LA, USA)

Dong, L. and Lapata, M. (2016). Language to logical form with neural attention. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* (Association for Computational Linguistics), 33–43

Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., et al. (2023). Pal: Program-aided language models. In *International Conference on Machine Learning* (PMLR), 10764–10799

Gardner, M., Artzi, Y., Basmov, V., Berant, J., Bogin, B., Chen, S., et al. (2020). Evaluating models' local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, eds. T. Cohn, Y. He, and Y. Liu (Online: Association for Computational Linguistics), 1307–1323. doi:10.18653/v1/2020.findings-emnlp.117

Goldberg, A. E. (1995). *Constructions: A Construction Grammar Approach to Argument Structure* (University of Chicago Press)

Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., et al. (2016). Hybrid computing using a neural network with dynamic external memory. *Nature* 538, 471–476

Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., et al. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*

Guo, Y., Lin, Z., Lou, J.-G., and Zhang, D. (2020). Hierarchical poset decoding for compositional generalization in language. *Advances in Neural Information Processing Systems* 33, 6913–6924

Gururangan, S., Swayamdipta, S., Levy, O., Schwartz, R., Bowman, S., and Smith, N. A. (2018). Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, eds. M. Walker, H. Ji, and A. Stent (New Orleans, Louisiana: Association for Computational Linguistics), 107–112. doi:10.18653/v1/N18-2017

Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*

Jia, R. and Liang, P. (2016). Data recombination for neural semantic parsing. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Association for Computational Linguistics)

Kautz, H. A. (2022). The third AI summer: AAAI Robert S. Engelmore Memorial Lecture. *AI Magazine* 43, 105–125. doi:10.1002/aaai.12036

Kazoom, R., Cohen, O., Puzis, R., Shabtai, A., and Hadar, O. (2025). VAULT: Vigilant adversarial updates via LLM-driven retrieval-augmented generation for NLI. *arXiv preprint arXiv:2508.00965*

Kempson, R., Meyer-Viol, W., and Gabbay, D. (2001). *Dynamic Syntax: The Flow of Language Understanding* (Blackwell)

Keysers, D., Schärli, N., Scales, N., Buisman, H., Furrer, D., Kashubin, S., et al. (2020). Measuring compositional generalization: A comprehensive method on realistic data. In *International Conference on Learning Representations*

Kim, N. and Linzen, T. (2020). COGS: A compositional generalization challenge based on semantic interpretation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, eds. B. Webber, T. Cohn, Y. He, and Y. Liu (Online: Association for Computational Linguistics), 9087–9105. doi:10.18653/v1/2020.emnlp-main.731

Klinger, T., Liu, L., Dan, S., Crouse, M., Ram, P., and Gray, A. (2023). Compositional program generation for few-shot systematic generalization. *arXiv preprint arXiv:2309.16467*

Kogkalidis, K., Moortgat, M., and Moot, R. (2020). Neural proof nets. In *Proceedings of the 24th Conference on Computational Natural Language Learning* (Association for Computational Linguistics), 26–40

Lake, B. M. and Baroni, M. (2018). Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *Proceedings of the 35th International Conference on Machine Learning* (PMLR), vol. 80 of *Proceedings of Machine Learning Research*, 2873–2882

Lake, B. M. and Baroni, M. (2023). Human-like systematic generalization through a meta-learning neural network. *Nature* 623, 115–121. doi:10.1038/s41586-023-06668-3

Lappin, S. (2021). *Deep Learning and Linguistic Representation* (Boca Raton, FL: CRC Press), 1st edn. doi:10.1201/9781003127086

Lappin, S. (2025). Neuro-symbolic models of AI. *Journal of Applied Logics* 12, 1631–1642

Li, Z., Huang, J., and Naik, M. (2023). Scallop: A language for neurosymbolic programming. *Proceedings of the ACM on Programming Languages* 7, 1463–1487

Li, Z., Zhou, Z., Yao, Y., Li, Y.-F., Cao, C., Yang, F., et al. (2024). Neuro-symbolic data generation for math reasoning. In *Advances in Neural Information Processing Systems* (Curran Associates, Inc.), vol. 37

Linzen, T. and Baroni, M. (2021). Syntactic structure from deep learning. *Annual Review of Linguistics* 7, 195–212. doi:10.1146/annurev-linguistics-032020-051035

Liu, H., Liu, J., Cui, L., Teng, Z., Duan, N., Zhou, M., et al. (2023). LogiQA 2.0—An improved dataset for logical reasoning in natural language understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 31, 2947–2962. doi:10.1109/TASLP.2023.3293046

Lu, X., West, P., Zellers, R., Le Bras, R., Bhagavatula, C., and Choi, Y. (2021). NeuroLogic decoding: (un)supervised neural text generation with predicate logic constraints. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, eds. K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou (Online: Association for Computational Linguistics), 4288–4299. doi:10.18653/v1/2021.naacl-main.339

Manhaeve, R., Dumančić, S., Kimmig, A., Demeester, T., and De Raedt, L. (2021). Neural probabilistic logic programming in deepproblog. *Artificial Intelligence* 298, 103504

Mao, J., Gan, C., Kohli, P., Tenenbaum, J. B., and Wu, J. (2019). The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. In *International Conference on Learning Representations (ICLR)*

Minervini, P., Bošnjak, M., Rocktäschel, T., Riedel, S., and Grefenstette, E. (2020). Differentiable reasoning on large knowledge bases and natural language. In *Proceedings of the AAAI conference on artificial intelligence*. vol. 34, 5182–5190

Monroe, W., Hawkins, R. X., Goodman, N. D., and Potts, C. (2017). Colors in context: A pragmatic neural model for grounded language understanding. *Transactions of the Association for Computational Linguistics* 5, 325–338

Nguyen, T. H. and Grishman, R. (2015). Relation extraction: Perspective from convolutional neural networks. In *Proceedings of the 1st Workshop on Vector Space Modeling for Natural Language Processing*, eds. P. Blunsom, S. Cohen, P. Dhillon, and P. Liang (Denver, Colorado: Association for Computational Linguistics), 39–48. doi:10.3115/v1/W15-1506

Nie, Y., Williams, A., Dinan, E., Bansal, M., Weston, J., and Kiela, D. (2020). Adversarial NLI: A new benchmark for natural language understanding. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, eds. D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault (Online: Association for Computational Linguistics), 4885–4901. doi:10.18653/v1/2020.acl-main.441

Olausson, T. X., Gu, A., Lipkin, B., Zhang, C. E., Solar-Lezama, A., Tenenbaum, J. B., et al. (2023). LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (Singapore: Association for Computational Linguistics), 5153–5176

Pan, L., Albalak, A., Wang, X., and Wang, W. Y. (2023). Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023* (Singapore: Association for Computational Linguistics), 3806–3824

Patel, A., Bhattamishra, S., Blunsom, P., and Goyal, N. (2022). Revisiting the compositional generalization abilities of neural sequence models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 424–434

Pavlick, E. (2022). Semantic structure in deep learning. *Annual Review of Linguistics* 8, 447–471. doi:10.1146/annurev-linguistics-031120-122924

Peng, B., Narayanan, S., and Papadimitriou, C. (2024). On limitations of the transformer architecture. *Collegium Beatus Rhenanus*

Qu, M., Chen, J., Xhonneux, L.-P., Bengio, Y., and Tang, J. (2021). Rnnlogic: Learning logic rules for reasoning on knowledge graphs. In *International Conference on Learning Representations (ICLR)*

Quan, X., Valentino, M., Dennis, L. A., and Freitas, A. (2024). Verification and refinement of natural language explanations through LLM-symbolic theorem proving. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, eds. Y. Al-Onaizan, M. Bansal, and Y.-N. Chen (Miami, Florida, USA: Association for Computational Linguistics), 2933–2958. doi:10.18653/v1/2024.emnlp-main.172

Riegel, R., Gray, A., Luus, F., Khan, N., Makondo, N., Akhalwaya, I. Y., et al. (2020). Logical neural networks. *arXiv preprint arXiv:2006.13155*

Rocktäschel, T. and Riedel, S. (2017). End-to-end differentiable proving. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. 3791–3803

Sachan, D., Zhang, Y., Qi, P., and Hamilton, W. L. (2021). Do syntax trees help pre-trained transformers extract information? In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 2647–2661

Saha, S., Ghosh, S., Srivastava, S., and Bansal, M. (2020). Prover: Proof generation for interpretable reasoning over rules. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 122–136

Sanwal, M. (2024). Evaluating large language models using contrast sets: An experimental approach. *arXiv preprint arXiv:2404.01569*

Sarker, M. K., Zhou, L., Eberhart, A., and Hitzler, P. (2021). Neuro-symbolic artificial intelligence: Current trends. *AI Communications* 34, 197–209. doi:10.3233/AIC-210084

Serafini, L. and Garcez, A. d. (2016). Logic tensor networks: Deep learning and logical reasoning from data and knowledge

Smolensky, P. (1990). Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence* 46, 159–216. doi:10.1016/0004-3702(90)90007-M

Tafjord, O., Dalvi, B., and Clark, P. (2021). ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, eds. C. Zong, F. Xia, W. Li, and R. Navigli (Online: Association for Computational Linguistics), 3621–3634. doi:10.18653/v1/2021.findings-acl.317

Tai, K. S., Socher, R., and Manning, C. D. (2015). Improved semantic representations from tree-structured long short-term memory networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, eds. C. Zong and M. Strube (Beijing, China: Association for Computational Linguistics), 1556–1566. doi:10.3115/v1/P15-1150

Teng, Z., Ning, R., Liu, J., Zhou, Q., Zhang, Y., et al. (2023). Glore: Evaluating logical reasoning of large language models. *arXiv preprint arXiv:2310.09107*

Tian, Y., Song, Y., Xia, F., and Zhang, T. (2020). Supertagging combinatory categorial grammar with attentive graph convolutional networks. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (Association for Computational Linguistics), 6037–6044

Trinh, T. H., Wu, Y., Le, Q. V., He, H., and Luong, T. (2024). Solving olympiad geometry without human demonstrations. *Nature* 625, 476–482. doi:10.1038/s41586-023-06747-5

Wang, H., Xin, H., Zheng, C., Li, L., Liu, Z., Cao, Q., et al. (2024a). LEGO-Prover: Neural theorem proving with growing libraries. In *The Twelfth International Conference on Learning Representations*. Oral presentation

Wang, S., Wei, Z., Choi, Y., and Ren, X. (2024b). Symbolic working memory enhances language models for complex rule application. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, eds. Y. Al-Onaizan, M. Bansal, and Y.-N. Chen (Miami, Florida, USA: Association for Computational Linguistics), 17583–17604. doi:10.18653/v1/2024.emnlp-main.974

Weber, L., Minervini, P., Münchmeyer, J., Leser, U., and Rocktäschel, T. (2019). NLProlog: Reasoning with weak unification for question answering in natural language. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (Florence, Italy: Association for Computational Linguistics), 6151–6161. doi:10.18653/v1/P19-1618

Xu, Z., Shi, Z., and Liang, Y. (2024). Do large language models have compositional ability? an investigation into limitations and scalability. In *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*

Yanaka, H., Mineshima, K., Bekki, D., Inui, K., Sekine, S., Abzianidze, L., et al. (2019). Can neural networks understand monotonicity reasoning? In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP* (Association for Computational Linguistics)

Yang, Z., Ishay, A., and Lee, J. (2020). Neurasp: Embracing neural networks into answer set programming. In *29th International Joint Conference on Artificial Intelligence (IJCAI 2020)*

Ye, X., Chen, Q., Dillig, I., and Durrett, G. (2023). Satlm: Satisfiability-aided language models using declarative prompting. *Advances in Neural Information Processing Systems* 36, 45548–45580

Yi, K., Wu, J., Gan, C., Torralba, A., Kohli, P., and Tenenbaum, J. B. (2018). Neural-symbolic VQA: disentangling reasoning from vision and language understanding. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. 1039–1050

Yogatama, D., Blunsom, P., Dyer, C., Grefenstette, E., and Ling, W. (2017). Learning to compose words into sentences with reinforcement learning. In *5th International Conference on Learning Representations (ICLR 2017)* (Toulon, France)

Zeng, D., Liu, K., Lai, S., Zhou, G., and Zhao, J. (2014). Relation classification via convolutional deep neural network. In *Proceedings of COLING 2014, the 25th international conference on computational linguistics: technical papers*. 2335–2344

Zhang, H., Huang, J., Li, Z., Naik, M., and Xing, E. (2023). Improved logical reasoning of language models via differentiable symbolic programming. In *Findings of the Association for Computational Linguistics: ACL 2023*, eds. A. Rogers, J. Boyd-Graber, and N. Okazaki (Toronto, Canada: Association for Computational Linguistics), 3062–3077. doi:10.18653/v1/2023.findings-acl.191

Zhang, T., Ge, J., Luo, H., Chuang, Y.-S., Gao, M., Gong, Y., et al. (2024). Natural language embedded programs for hybrid language symbolic reasoning. In *Findings of the Association for Computational Linguistics: NAACL 2024*, eds. K. Duh, H. Gomez, and S. Bethard (Mexico City, Mexico: Association for Computational Linguistics), 4131–4155. doi:10.18653/v1/2024.findings-naacl.259

Zhou, J. and Zhao, H. (2019). Head-driven phrase structure grammar parsing on Penn treebank. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (Florence, Italy: Association for Computational Linguistics), 2396–2408